SIX REPOS, ONE SYSTEM

forecasting. portfolio. execution. market making. research. every job a desk pays a person to do, open source, running on your own machine.

---

this is a reading list, not a product. six repositories, what each one is actually for, what it is not, and the order they connect in. the links are below each name. everything here is free and everything here is for research and paper trading first.

THE RULE THAT MATTERS MOST

the model forecasts. the optimizer sizes. the engine executes. the language model never touches the order path. keep those four jobs separate and the stack stays debuggable. blur them and you have a black box you cannot audit and cannot fix.

01	kronos, forecasting from raw candles
02	skfolio, portfolio and risk sizing
03	nautilus trader, execution engine
04	hummingbot, market making and arbitrage
05	freqtrade, the full retail bot loop
06	qlib, research and factor pipeline

READ FIRST

how to use this list

this document is educational. it is not personalised financial advice and nothing in it is a recommendation to trade, to buy any asset, or to run any of this software with real money. open source trading software is powerful and unforgiving. bugs, bad data and wrong assumptions cost real capital. default to historical replay and paper trading, and read the license and the docs of every repo before you run anything.

THE STACK, IN ORDER

a) **data** · candles, order book, fundamentals

b) **forecast** · a model that outputs a distribution, not a signal

c) **portfolio** · how much, given risk and correlation

d) **execution** · orders, fills, slippage, reconnection

e) **research** · the loop that tells you whether any of it held up

WHAT EACH LAYER IS ALLOWED TO DECIDE

LAYER	DECIDES
forecast	the probability of something, with uncertainty attached. never the size, never the order
portfolio	the weight, given risk, correlation and constraints you wrote in advance
execution	how the order reaches the venue, and what happens when it does not
research	whether the previous three earned the right to keep running

WHY THE SEPARATION IS THE WHOLE POINT

an end to end model that takes candles in and sends orders out cannot be debugged. when it stops working, and it will, you have no idea which part broke. four separate components with defined inputs and outputs can each be tested, replaced and killed independently. that is not academic tidiness, it is the difference between a system and a slot machine.

star counts, licenses and features change. verify on the repository before you build on any of it. links are to the official repos as of writing.

THE STACK

forecast and portfolio

01 KRONOS

PYTHON · MIT

a foundation model trained on the language of candles. a tokenizer turns OHLCV sequences into discrete tokens, and a decoder only transformer is pre-trained on them across dozens of global exchanges. you feed it a K line history and it returns a probabilistic forecast, which means a range with uncertainty attached rather than an arrow.

github.com/shiyu-coder/Kronos

use it for	price and volatility forecasting, synthetic data, representation learning on raw candles
not for	signals. it does not tell you to buy anything, and treating its mean output as a signal is the fastest way to misuse it

LAYER: FORECAST

02 SKFOLIO

PYTHON · BSD-3

portfolio optimization and risk management built directly on the scikit learn API. same fit, predict, transform pattern, so cross validation, hyperparameter search and pipeline composition work exactly as they do in any ML workflow. mean variance, risk parity, hierarchical clustering, factor models, stress testing, and cross validation designed for financial time series rather than shuffled rows.

github.com/skfolio/skfolio

use it for	turning a set of signals into weights, with the leakage and overfitting controls built in
not for	generating alpha. it allocates what you give it, and it will happily optimize garbage into a confident looking allocation

LAYER: PORTFOLIO

THE STACK

execution and market making

03 NAUTILUS TRADER

RUST CORE · PYTHON API · LGPL

an event driven trading platform with a Rust core and a Python API. the point of the project is parity: the same strategy code runs in backtest and in live, against the same event model, so the gap between what you tested and what you deployed is as small as software can make it. multi venue, multi asset, built for correctness under load rather than for a pretty notebook.

github.com/nautechsystems/nautilus_trader

use it for	event driven backtests that behave like live, and the execution layer underneath a serious strategy
not for	a quick idea test. the learning curve is real and the abstractions cost you a weekend before they pay

LAYER: EXECUTION

04 HUMMINGBOT

PYTHON · APACHE 2.0

an open source framework for market making and arbitrage across a long list of centralised and decentralised venues. connectors are the product: standardised access to order books, balances and orders across exchanges, with strategy templates for the classic inventory and spread problems on top.

github.com/hummingbot/hummingbot

use it for	studying inventory risk, spreads and adverse selection with real venue mechanics instead of theory
not for	passive income. market making is a risk transfer business, and running it badly transfers risk toward you

LAYER: EXECUTION / VENUE

THE STACK

the loop and the research

05 FREQTRADE

PYTHON · GPL-3

the complete retail crypto bot, and the most approachable entry point on this list. strategies in plain Python, backtesting and hyperparameter optimization built in, dry run mode by default, control from telegram or a web UI, and FreqAI on top for adaptive models that retrain on a rolling window instead of being fitted once and trusted forever.

github.com/freqtrade/freqtrade

use it for	learning the whole loop end to end: strategy, backtest, dry run, monitoring, iteration
not for	skipping the study. hyperopt will find you a beautiful curve fitted strategy in an afternoon if you let it

LAYER: FULL LOOP

06 QLIB

PYTHON · MIT

microsoft's AI oriented quantitative research platform. a data layer, a factor and feature pipeline, a model zoo, and a backtest and analysis stage, wired together as one workflow so an experiment is reproducible rather than a notebook nobody can rerun. the companion project RD-Agent pushes further, using language models to automate parts of the research loop itself, proposing and testing factors and models.

github.com/microsoft/qlib · github.com/microsoft/RD-Agent

use it for	the research layer: factors, model comparison, walk forward evaluation, reproducible experiments
not for	plugging into a broker tomorrow. it is a research platform, and it expects you to prepare data properly

LAYER: RESEARCH

THE DISCIPLINE

keep the model out of the order path

the most common failure with this stack is not a bad model. it is wiring a language model directly to an execution client because it works in a demo. it will work for a while, and then it will do something you cannot explain at a moment you cannot afford.

ALLOWED	NOT ALLOWED
a model proposes a factor, a feature, a hypothesis	a model sends an order
a model writes strategy code you read before running	a model writes and deploys code unreviewed
a model summarises a backtest you can reproduce	a model reports results you cannot verify
a model audits your log and finds repeats	a model decides risk, size or exits at runtime

THE PAPER TRADING PROTOCOL

1. run everything in replay or dry run first. no exceptions, no "just small size to see"
2. choose the test period before you look at it. include a trending phase, a ranging phase, a high volatility phase
3. add costs, slippage and a realistic fill assumption. an edge that dies at one tick of slippage was never an edge
4. delete your five best trades and check whether the result survives
5. keep a holdout period untouched. test it once, at the end, and accept what it says
6. write kill criteria before you start, in numbers, so you cannot argue with yourself later

THE PART THAT DECIDES EVERYTHING

none of these repos fix the thing that actually loses money, which is an unspecified process. a bot is a specification with a runtime attached. if you cannot write your rules as conditions a machine could evaluate, you do not have a strategy to automate yet, and automating it will simply lose money faster and more consistently.

BEFORE YOU BUILD

the part the repos assume you already have

every project on this list starts at the same place: you hand it a rule. kronos needs to know what you are forecasting and why. skfolio needs constraints you chose in advance. nautilus needs a strategy class. freqtrade needs entry and exit conditions in Python. qlib needs a hypothesis worth testing.

none of them will tell you what your edge is. that part is upstream of all the code, and it is the part most people skip, which is why so many repos get cloned, run once, and abandoned within a week.

START WITH THE PROCESS, THEN AUTOMATE IT

the 5 day reset	free. five sessions of structured logging, then five prompts that turn your own log into written conditions. this is how you find out whether you have anything worth coding
the daily bias guide	free. the four inputs read before every session, written before the open and never edited after
the vault	the full framework: 12 modules, 33 lessons, linked note to note in obsidian. liquidity, structure, confirmation, correlation, risk, and the mindset layer that decides whether any of it survives a losing week. one time, no subscription

no countdown and no fake deadline. the only cost of waiting is another quarter spent rediscovering rules that were already written down.

THE FRAMEWORK BEHIND THE CODE

payoutvault.one

free guides and the full vault in one place · @iclosegreen